
CSE234 Project2 Report

1 Training Data Methodology

1.1 Training Data Composition

The training data was constructed from the provided schema-linking dataset. Each record contains a question identifier (`question_id`), a database identifier (`db_id`), the text of the question, a gold SQL query, and a ground-truth `schema_links` annotation.

For training, we used only the `db_id`, `question`, and `schema_links` fields. The `db_id` was used to retrieve the corresponding database schema, the text of the question served as the user query, and the `schema_links` annotation was used as the target output. The `gold_sql` and `question_id` fields were not used during training.

The original training set contains 301 examples, while the validation set contains 101 examples.

1.2 Input and Output Serialization

We formulate schema linking as a chat-style supervised fine-tuning task. Each training example is converted into a sequence of system, user, and assistant messages.

The system message defines the schema-linking task and instructs the model to generate only valid JSON outputs. The user message contains the database schema and the text of the question. Database schemas are serialized into a compact text format where each table is represented together with its associated columns. The assistant message contains the ground-truth `schema_links` annotation encoded as a JSON object. This formulation allows the model to learn a direct mapping from a question and its database schema to the corresponding schema links.

System:

You are a schema linking model for text-to-SQL.
Given a natural language question and a database schema,
return ONLY valid JSON.

The JSON format must be:

```
{"TableName": ["Column1", "Column2"], "AnotherTable": []}
```

Rules:

- Use only table and column names from the provided schema.
- Include a table with `[]` if the table is referenced but no specific column is needed.
- Do not explain.
- Do not output markdown.

User:

Database: NTSB

Schema:

- AIRCRAFT(...)
- OCCURRENCE(...)
- INJURY(AIS, REGION, ...)
- ...

Question:

Show a count of injuries by body region where the injury severity is critical. The lookup code for critical injury is 5.

Return `schema_links` JSON only.

```
Assistant:
{"INJURY": ["AIS", "REGION"]}
```

1.3 Data Augmentation

We did not apply synthetic data augmentation since the dataset is relatively small and consists of manually curated schema-link annotations. Generating artificial training examples could introduce incorrect schema associations and reduce label quality.

1.4 Curriculum / balancing

We applied a limited oversampling strategy during training. Examples from the SBODemoUS database were duplicated once because preliminary inspection of the training set showed that this database appeared less frequently than several other databases. In addition, examples involving multiple linked tables were also duplicated because multi-table schema linking is generally more challenging than single-table linking. This increases the effective weight of underrepresented schemas and encourages the model to better learn complex cross-table relationships while preserving the original annotations.

1.5 Diagnostic Metrics

We defined an additional diagnostic metric, `exact_json_rate`, which measures the percentage of predictions whose schema-link JSON exactly matches the ground-truth schema-link annotation after JSON parsing. This metric provides a simple and efficient way to compare configurations during RapidFireAI experiments.

However, `exact_json_rate` was not used as the final evaluation criterion. Although we implemented this metric for diagnostic purposes, the RapidFireAI training pipeline does not directly report its evaluation results in a way that can be used for final model comparison. Therefore, for official evaluation, we generated predictions on the validation set and ran the provided `eval.py` script, which computes the required table-level and column-level precision, recall, and F1 metrics. All reported final results were obtained using `eval.py` rather than `exact_json_rate`.

2 Final Pipeline Architecture

The final pipeline uses Qwen/Qwen3-1.7B as the base model. This model satisfies the project constraint of using a model under 2B parameters, while still giving more capacity than the smaller 0.5B baselines. We fine-tuned it with a PEFT LoRA adapter.

The final LoRA configuration uses rank $r = 16$, $\alpha = 32$, and `lora_dropout=0.05`. The learning rate is set to $3e - 4$. We apply LoRA to the main attention projection layers: `q_proj`, `k_proj`, `v_proj`, and `o_proj`. We set `bias="none"`, so only the LoRA adapter weights are trained. This configuration gives the adapter more capacity than a smaller Q/V-only LoRA setup, while still keeping training and inference lightweight.

The prompt is a chat-style prompt with a system message and a user message. The system message tells the model that it is a schema linking model for text-to-SQL and that it must return only valid JSON in the required format:

```
{"TableName": ["Column1", "Column2"], "AnotherTable": [ ]}
```

The user prompt contains the database id, a serialized schema, the natural language question, and a final instruction to return only the `schema_links` JSON. The schema is serialized in a compact text format:

```
Database: <db_id>
Schema:
- TableA(col1, col2, col3)
- TableB(col1, col2)

Question:
<question>

Return schema_links JSON only.
```

During training, the assistant response is the gold schema_links object serialized as compact JSON. This makes the training target match the final evaluation format directly. At inference time, `main.py` loads the tokenizer and base causal language model with HuggingFace transformers, then loads the trained adapter with PEFT. The prompt messages are converted into the model’s chat format using `tokenizer.apply_chat_template()`. Generation is deterministic: `do_sample=False`, `max_new_tokens=256`, and `repetition_penalty=1.02`. After generation, the original prompt tokens are removed and only the newly generated tokens remain.

The post-processing step is designed to make the output valid for the evaluator. First, we try to parse the whole model output as JSON. If that fails, we remove possible markdown code fences and then search for a JSON object inside the text. If no valid JSON dictionary can be parsed, an empty prediction is returned instead of crashing.

After JSON extraction, the pipeline validates all predicted identifiers against the actual schema for that db_id. Table and column matching is case-insensitive, but the final output restores the original casing from the schema files. Any hallucinated table name is dropped. Any hallucinated column name, or any column that does not belong to the predicted table, is also dropped. The remaining columns are deduplicated and sorted for stable output. This post-processing reduces invalid identifiers and makes the final prediction follow the required schema-linking JSON format.

3 Experimentation Methodology

3.1 Multi-Config Combinations

RapidFireAI provides the RFGGridSearch API for evaluating multiple configurations in a single experiment. We used this functionality whenever the configuration differences could be expressed through parameter settings, such as LoRA hyperparameters and base model training parameters. However, some experimental changes were not easily represented through RFGGridSearch. For example, schema serialization strategies required modifications to the data preprocessing and prompt construction pipeline. In these cases, we trained separate SFT models and compared their performance manually. For each configuration, a fine-tuned model was trained and used to generate predictions on the validation set. Although RapidFireAI provides built-in evaluation metrics, our final model comparison was based on the official `eval.py` evaluation script to ensure consistency with the course evaluation procedure. We evaluated multiple configuration dimensions, including schema serialization, output format, curriculum and balancing strategies, base model selection, and adapter hyperparameters. The explored configuration knobs are summarized in Table 1.

Table 1: Configuration settings

Configuration	Settings
Schema serialization	Flat Table-Column; Flat Table-Column + PK/FK; Flat Table-Column + BM25 Filtering
Output format	JSON Only; Sorted JSON; Table-Column + Sorted JSON; Intermediate Reasoning + Table-Column + JSON; Intermediate Reasoning + JSON
Data augmentation	None
Curriculum / balancing	None; Oversampling; Oversampling + Curriculum
LoRA rank	8, 16
LoRA alpha	16, 32
Target modules	q_proj,v_proj; q_proj,k_proj,v_proj,o_proj
Base model	Qwen2.5-0.5B; Qwen2.5-1.5B; Qwen3-1.7B
Learning rate	1e-4, 2e-4, 3e-4, 5e-4
LR scheduler type	linear, cosine
Number of epochs	3, 4

We explored several schema serialization strategies to study how schema representation affects schema-link prediction. The baseline configuration used a flat table-column serialization format. We additionally evaluated schema representations augmented with primary and foreign key information, since explicit relational information may help the model better understand table relationships. We also explored BM25-based schema filtering to reduce prompt length and remove irrelevant tables, with the goal of focusing the model’s attention on potentially relevant schema elements.

Different output formats were investigated to determine whether providing intermediate structure or reasoning signals could improve learning. In addition to direct JSON prediction, we evaluated sorted JSON outputs, table-

column representations, and intermediate reasoning formats that explicitly expose relevant tables and columns before producing the final JSON output.

For data-level strategies, we compared standard training, oversampling-based balancing, and balancing combined with curriculum learning. Oversampling was introduced to increase the influence of underrepresented databases and more complex multi-table examples. Curriculum learning further orders training examples from easier to harder instances based on schema-link complexity.

For adapter configurations, we evaluated different LoRA ranks, LoRA scaling factors, and target modules. Higher LoRA ranks increase adaptation capacity at the cost of additional trainable parameters, while different target modules allow adaptation of varying portions of the attention mechanism.

We also compared multiple base models, including Qwen2.5-0.5B, Qwen2.5-1.5B, and Qwen3-1.7B, to evaluate the impact of model scale. Finally, learning rates, scheduler types, and training epochs were varied to identify stable optimization settings and avoid both underfitting and overfitting.

3.2 Meaningfully Distinct Configs

We evaluated 8 meaningfully distinct configurations of the schema-linking pipeline by varying schema serialization strategies, output formats, curriculum and balancing strategies, base model configurations, and adapter configurations, as summarized in Table 2. We first adapted the provided starter code and verified the complete training and evaluation pipeline using a lightweight base model (Qwen2.5-0.5B) together with a simple flat table-column schema representation and direct JSON prediction. This configuration established a reference point for all subsequent experiments, which is experiment **C1**, and served as our baseline. Then we changed the schema serialization strategy to Flat Table-Column with PK/FK information since explicitly exposing relational structure could help the model better understand table relationships and improve schema-link prediction for multi-table queries, which is experiment **C2**. We also tried another strategy, BM25 Filtering Flat Table-Column, so that it included only a subset of schema tables that were most relevant to the input question. This was motivated by the observation that large schemas increase prompt length and may distract the model with irrelevant schema information, which is experiment **C3**. After modifying Schema Serialization, we started adding Curriculum or Balancing strategies. We introduced oversampling for underrepresented databases and complex multi-table examples since this balancing strategy could increase the influence of difficult training instances and improve generalization, which is experiment **C4**. We further incorporated curriculum learning on top of **C4** by sorting training examples according to a manually designed difficulty score based on the number of linked tables, linked columns, and question length, which exposes the model to simpler examples before progressively introducing more complex schema-linking cases, which is experiment **C5**. After identifying balancing as one of the most effective modifications, we fixed the balancing strategy and evaluated a larger LoRA configuration by increasing both the LoRA rank and LoRA alpha, since additional adaptation capacity could improve schema-link prediction performance, which is experiment **C6**. We then replaced the baseline Qwen2.5-0.5B model with Qwen2.5-1.5B while keeping the remaining settings unchanged since a larger instruction-tuned language model could provide stronger schema understanding and reasoning capabilities, which is experiment **C7**. After evaluating the effect of model scale, we further tried a different output format. Instead of directly generating the final JSON output, the model was trained to first produce intermediate schema-link reasoning and then generate the final JSON representation. We hypothesized that explicitly exposing relevant schema elements during generation could encourage more structured reasoning and improve prediction accuracy, which is experiment **C8**.

Table 2: 8 meaningful configuration settings were explored during schema-linking fine-tuning experiments.

ID	Schema Serialization	Output Format	Augmentation	Curriculum / Balancing	Rank	Alpha	Base Model
C1	Flat Table-Column	JSON Only	None	None	8	16	Qwen2.5-0.5B
C2	Flat Table-Column + PK/FK	JSON Only	None	None	8	16	Qwen2.5-0.5B
C3	Flat Table-Column + BM25 Filtering	JSON Only	None	None	8	16	Qwen2.5-0.5B
C4	Flat Table-Column	JSON Only	None	Oversampling	8	16	Qwen2.5-0.5B
C5	Flat Table-Column	JSON Only	None	Oversampling + Curriculum	8	16	Qwen2.5-0.5B
C6	Flat Table-Column	JSON Only	None	Oversampling	16	32	Qwen2.5-0.5B
C7	Flat Table-Column	JSON Only	None	Oversampling	16	32	Qwen2.5-1.5B
C8	Flat Table-Column	Intermediate Reasoning + JSON	None	Oversampling	16	32	Qwen2.5-1.5B

4 Results and Analysis

4.1 Schema Serialization

For schema serialization, the initial version employed a Flat Table-Column representation, which served as our baseline (**C1**). We compared this baseline with two alternative schema serialization strategies: Flat Table-Column with PK/FK information (**C2**) and Flat Table-Column with BM25 Filtering (**C3**), which were intended to provide additional relational information and reduce irrelevant schema content respectively. The experimental results are shown in Table 3. Among the three configurations, the baseline Flat Table-Column representation achieved the highest leaderboard score (0.2224), outperforming both the PK/FK-enhanced version (0.2182) and the BM25-filtered version (0.1808). Although adding PK/FK information slightly improved column-level precision (0.1971 vs. 0.1716) and column-level F1 (0.1591 vs. 0.1364), all table-level metrics decreased compared with the baseline. This suggests that the additional PK/FK information may have helped the model identify relevant columns once the correct tables were selected, but the increased schema complexity and prompt length made it more difficult for the model to identify the correct tables. As a result, the overall leaderboard score did not improve. For BM25-based schema filtering, both table-level and column-level metrics decreased substantially. In particular, the table-level F1 score dropped from 0.2702 to 0.2264, while the column-level F1 score dropped from 0.1364 to 0.1061. A possible explanation is that schema linking requires access to all potentially relevant schema elements, whereas BM25 filtering may remove tables that are semantically relevant but do not share strong lexical overlap with the question. Consequently, the model may lose access to useful schema information, resulting in lower recall and overall performance. Since neither PK/FK augmentation nor BM25-based schema filtering improved the leaderboard score, we retained the original Flat Table-Column serialization strategy in the final pipeline.

Table 3: Results of different schema serialization strategies.

ID	Schema Serialization	Table				Column				LB
		Prec.	Rec.	F1	Score	Prec.	Rec.	F1	Score	
C1	Flat Table-Column	0.3114	0.2706	0.2702	0.2841	0.1716	0.1745	0.1364	0.1608	0.2224
C2	Flat Table-Column + PK/FK	0.2896	0.2492	0.2545	0.2644	0.1971	0.1599	0.1591	0.1720	0.2182
C3	Flat Table-Column + BM25 Filtering	0.2772	0.2071	0.2264	0.2369	0.1435	0.1247	0.1061	0.1248	0.1808

4.2 Output Format

For output format design, we compared five different output representations: JSON, Sorted JSON, Table-Col + Sorted JSON, Reasoning + Table-Col + JSON, and Reasoning + JSON (Experiment **C8**). The experimental results are shown in Table 4. Among all five configurations, the original JSON-only output format achieved the highest leaderboard score (0.5155). Compared with the baseline, Sorted JSON reduced the leaderboard score from 0.5155 to 0.4917, while Table-Col + Sorted JSON further reduced it to 0.4887. This suggests that introducing additional formatting constraints or alternative schema-link representations did not improve the model’s ability to predict schema links. The two reasoning-based output formats also failed to improve performance. Although Reasoning + Table-Col + JSON achieved a column-level F1 score (0.3998) that was nearly identical to the baseline (0.3999), its table-level F1 score decreased from 0.5865 to 0.5329, resulting in a lower leaderboard score (0.4829). Similarly, Reasoning + JSON produced the lowest leaderboard score (0.4601), with both table-level F1 (0.5310) and column-level F1 (0.3480) decreasing substantially compared with the baseline. These results suggest that schema linking is fundamentally a structured prediction task and does not benefit from additional intermediate structures or reasoning steps. While the reasoning-based formats introduce extra information into the output, the evaluation script only considers the final schema links, meaning that the additional reasoning is not directly rewarded. Furthermore, generating intermediate reasoning increases output complexity and introduces additional opportunities for prediction errors. Therefore, we retained the original JSON-only output format in the final pipeline.

Table 4: Results of different output format strategies.

ID	Output Format	Table				Column				LB
		Prec.	Rec.	F1	Score	Prec.	Rec.	F1	Score	
1	JSON	0.6172	0.6073	0.5865	0.6036	0.4533	0.4287	0.3999	0.4273	0.5155
2	Sorted JSON	0.5688	0.5924	0.5534	0.5715	0.4104	0.4369	0.3886	0.4120	0.4917
3	Table-Col + Sorted JSON	0.5912	0.5817	0.5505	0.5744	0.4399	0.3883	0.3806	0.4029	0.4887
4	Reasoning + Table-Col + JSON	0.5670	0.5347	0.5329	0.5449	0.4410	0.4218	0.3998	0.4209	0.4829
C8	Reasoning + JSON	0.5586	0.5413	0.5310	0.5436	0.3833	0.3985	0.3480	0.3766	0.4601

¹ Note: Experiments 1–4 use the same training configuration and differ only in the output format. Experiment **C8** uses the same configuration as Experiment 4 except for the output format.

4.3 Training Data Strategy: Balancing & Curriculum

For the training data strategy, balancing had a clear positive effect, while adding curriculum did not improve the results. Experiment results are shown in Table 5. On Qwen2.5-0.5B with the lightweight LoRA adapter, the balanced setup reached a leaderboard score of about 0.28, while balancing plus curriculum dropped to about 0.24. The same pattern also appeared on Qwen2.5-1.5B. With the $r = 8$ adapter, balancing alone achieved about 0.4298 LB, while balancing plus curriculum stayed almost the same. With the stronger $r = 16$ adapter, balancing alone reaching about 0.5197, but adding curriculum reduced the score to about 0.4759. These results suggest that repeating underrepresented databases and harder multi-table examples helped the model learn more useful schema-linking patterns, but changing the training order did not help this task. One likely reason is that the dataset is small, so exposing the model more often to rare schemas mattered more than controlling the order of examples. Based on these results, we kept balancing in the final pipeline and did not use curriculum.

Table 5: Performance Comparison with Different Training Data Strategies

Strategy ¹	Model	Adapter config	Table				Column				LB
			Precision	Recall	F1	Score	Precision	Recall	F1	Score	
-	Qwen2.5-0.5B	$r = 8$ $\alpha = 16$	0.3114	0.2706	0.2702	0.2841	0.1716	0.1745	0.1364	0.1608	0.2224
B	Qwen2.5-0.5B	$r = 8$ $\alpha = 16$	0.3581	0.3771	0.3448	0.3600	0.2044	0.2208	0.1747	0.2000	0.2800
B+C	Qwen2.5-0.5B	$r = 8$ $\alpha = 16$	0.3185	0.3201	0.3008	0.3132	0.1857	0.1855	0.1511	0.1741	0.2436
B	Qwen2.5-1.5B	$r = 8$ $\alpha = 16$	0.5127	0.5314	0.4925	0.5122	0.3510	0.3698	0.3212	0.3473	0.4298
B+C	Qwen2.5-1.5B	$r = 8$ $\alpha = 16$	0.5177	0.5264	0.4910	0.5117	0.3423	0.3678	0.3170	0.3424	0.4270
B	Qwen2.5-1.5B	$r = 16$ $\alpha = 32$	0.6155	0.6172	0.5921	0.6083	0.4573	0.4323	0.4037	0.4311	0.5197
B+C	Qwen2.5-1.5B	$r = 16$ $\alpha = 32$	0.5644	0.5429	0.5273	0.5449	0.4088	0.4308	0.3815	0.4070	0.4759

¹ Note: "B" represents the Balancing strategy, "C" represents the Curriculum strategy.

4.4 Adapter Configuration

For the adapter configuration, we compared two LoRA settings on both Qwen2.5-0.5B and Qwen2.5-1.5B. The lighter adapter used $r = 8, \alpha = 16$, and only targeted `q_proj` and `v_proj`. The stronger adapter used $r = 16, \alpha = 32$, and targeted all four attention projection layers: `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The experiment results are shown in Table 6. Overall, the stronger adapter performed better, especially on column-level metrics. On Qwen2.5-0.5B, moving from the lighter adapter to the stronger adapter improved the leaderboard score from about 0.28 to 0.3145. The table precision slightly decreased, but table recall improved, and the column score increased from about 0.20 to 0.2568. This suggests that the larger adapter helped the small model recover more relevant identifiers, especially columns, even though it also introduced some extra table

predictions. On Qwen2.5-1.5B, the improvement was much clearer. The table score increased from 0.5122 to 0.6083, the column score increased from 0.3473 to 0.4311, and the leaderboard score increased from 0.4298 to 0.5197. This shows that the larger base model could make better use of the higher-rank adapter and the additional target modules. The results show that adapter capacity mattered a lot for this task. Schema linking is not only about following the JSON format, but also about selecting precise table-column pairs from a long schema. The stronger LoRA adapter gave the model more ability to adapt its attention layers to this identifier-selection task, which helped column-level performance the most. Based on these results, we selected the $r = 16, \alpha = 32$, q_proj/k_proj/v_proj/o_proj LoRA configuration for the final pipeline.

Table 6: Adapter Configuration Comparison ¹

Model	Adapter config	Table				Column				LB
		Precision	Recall	F1	Score	Precision	Recall	F1	Score	
Qwen2.5-0.5B	$r = 8$ $\alpha = 16$	0.3581	0.3771	0.3448	0.3600	0.2044	0.2208	0.1747	0.2000	0.2800
Qwen2.5-0.5B	$r = 16$ $\alpha = 32$	0.3401	0.4257	0.3507	0.3722	0.2491	0.2835	0.2377	0.2568	0.3145
Qwen2.5-1.5B	$r = 8$ $\alpha = 16$	0.5127	0.5314	0.4925	0.5122	0.3510	0.3698	0.3212	0.3473	0.4298
Qwen2.5-1.5B	$r = 16$ $\alpha = 32$	0.6155	0.6172	0.5921	0.6083	0.4573	0.4323	0.4037	0.4311	0.5197

¹ Note: Training data is augmented using "balancing" method.

4.5 Model Selection

For the base model comparison, we tested three models under the same stronger LoRA adapter setting: Qwen2.5-0.5B, Qwen2.5-1.5B, and Qwen3-1.7B. The adapter configuration was fixed to $r = 16, \alpha = 32$, lora_dropout = 0.05, and target modules q_proj, k_proj, v_proj, and o_proj. The learning rate was not exactly the same across all three runs. We used $2e - 4$ for Qwen2.5-0.5B, but used a smaller learning rate of $1e - 4$ for Qwen2.5-1.5B and Qwen3-1.7B. We made this choice because the larger models already have stronger language and instruction-following ability, so a smaller learning rate reduces the risk of overfitting or changing the base model behavior too aggressively.

The results in Table 7 show that model capacity had a large effect on performance. Qwen2.5-0.5B had the lowest leaderboard score, about 0.3145, with a table score of 0.3722 and a column score of 0.2568. After increasing the model size to Qwen2.5-1.5B, the leaderboard score improved to about 0.5197, with clear gains on both table score and column score. Qwen3-1.7B performed best overall, reaching a leaderboard score of about 0.5372. Compared with Qwen2.5-1.5B, its table score only improved slightly, from 0.6083 to 0.6131, but its column score improved more clearly, from 0.4311 to 0.4614. This was important because column linking is the harder part of the task and has a large impact on the final score. Based on these results, we selected Qwen3-1.7B as the final base model.

Table 7: Model Performance Comparison ¹

Model	Table				Column				LB
	Precision	Recall	F1	Score	Precision	Recall	F1	Score	
Qwen2.5-0.5B	0.3401	0.4257	0.3507	0.3722	0.2491	0.2835	0.2377	0.2568	0.3145
Qwen2.5-1.5B	0.6155	0.6172	0.5921	0.6083	0.4573	0.4323	0.4037	0.4311	0.5197
Qwen3-1.7B	0.5877	0.6683	0.5831	0.6131	0.4403	0.5182	0.4258	0.4614	0.5372

¹ Note: Training data is augmented using "balancing" method. We fix the adapter configuration as $r = 16, \alpha = 32$.

4.6 Learning Rate

For the learning rate comparison, we tested $1e - 4$, $3e - 4$, and $5e - 4$ on Qwen3-1.7B with the same final LoRA adapter configuration. The results were still very close overall, as shown in Table 8. The leaderboard scores were

about 0.5372 for $1e-4$, 0.5381 for $3e-4$, and 0.5331 for $5e-4$. Increasing the learning rate from $1e-4$ to $3e-4$ gave a very small improvement, mainly from column-level metrics, but increasing it further to $5e-4$ did not help. In fact, $5e-4$ slightly reduced the final leaderboard score, even though its column score was a little higher. This suggests that learning rate was not the main factor affecting performance after fixing the base model and adapter configuration. Compared with model size and LoRA capacity, learning rate behaved more like a secondary tuning knob, and $3e-4$ was the best among the values we tried.

Table 8: Learning Rate Performance Comparison ¹

Learning Rate	Table				Column				LB
	Precision	Recall	F1	Score	Precision	Recall	F1	Score	
$1e-4$	0.5877	0.6683	0.5831	0.6131	0.4403	0.5182	0.4258	0.4614	0.5372
$3e-4$	0.5904	0.6559	0.5899	0.6121	0.4467	0.5075	0.4385	0.4642	0.5381
$5e-4$	0.5552	0.6502	0.5743	0.5932	0.4600	0.5067	0.4523	0.4730	0.5331

¹ Note: Training data is augmented using "balancing" method. We fix the adapter configuration as $r = 16, \alpha = 32$. The model is Qwen3-1.7B.

4.7 Discussion and Future Work

Several knobs had a clear positive impact on the final leaderboard score. The most useful changes were the balancing-based training data strategy, the stronger LoRA adapter, and the larger base model. Balancing helped because the released training set was small and uneven across databases, so repeating underrepresented schemas and harder multi-table examples gave the model more chances to learn those patterns. The stronger adapter improved both table-level and column-level metrics, especially column linking, because targeting more attention projection layers gave the model more capacity to adapt to schema identifier selection. Increasing the base model size also helped both metrics, with the largest gain coming from moving away from the 0.5B model. In contrast, learning rate had only a small effect after the base model and adapter were fixed. The differences between the learning rate runs were small enough that they may partly come from normal training variance.

Some changes did not help as much as expected. We tested other training data strategies, including different schema serialization methods and different output formats, but they did not give clear improvements on this dataset. We also found that adding curriculum on top of balancing did not improve the result, and sometimes reduced performance. This was somewhat surprising, because we expected ordering examples by difficulty to make learning easier. A likely reason is that this task is more sensitive to coverage of rare databases and schema patterns than to the order in which examples are shown.

Overall, model size, adapter capacity, and training data balancing mattered more than learning rate tuning. Column-level metrics were harder to improve than table-level metrics, and the knobs that increased model capacity had the clearest effect on column F1. With more time and budget, we would try more data augmentation methods, such as paraphrasing natural language questions, synthesizing additional NL-SQL pairs, and adding negative examples. We would also compare other sub-2B model families, such as Llama-3.2-1B-Instruct and SmoLLM2-1.7B-Instruct, to see whether models with similar parameter counts behave differently on schema linking.

5 AI Tools Statement

We used AI assistants to support several parts of the project.

- AI tools were also used to assist with parts of the coding process, including debugging and implementation suggestions.
- AI assistants were used for report polishing and improving writing clarity.